Appels Réseau et Gestion de l'Asynchronisme

Objectifs

1. Comprendre le cycle de vie d'une requête HTTP au sein d'une application React.
2. Gérer correctement les différents états d'une requête (chargement, succès, erreur) dans l'état local.
3. Déporter la logique asynchrone dans l'état global avec Redux Toolkit (`createAsyncThunk`).
4. Prévenir les fuites de mémoire liées aux requêtes asynchrones sur des composants démontés.

1. Rappels sur les Appels Réseau en JavaScript

Dans une Single Page Application (SPA), le navigateur ne recharge jamais la page entière pour récupérer de nouvelles données. Il effectue des requêtes HTTP en arrière-plan vers une API (généralement REST ou GraphQL) et met à jour l'interface avec la réponse (format JSON).

Pour cela, JavaScript moderne utilise l'API native `fetch` ou des bibliothèques dédiées comme `axios`, combinées à la syntaxe `async / await`.

JavaScript

```
// Exemple classique d'une fonction asynchrone native
async function fetchStations() {
  try {
    const response = await fetch('https://api.omnimonitor.com/stations');
    if (!response.ok) {
      throw new Error('Erreur lors de la récupération des données');
    }
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Erreur réseau:', error);
  }
}
```

2. Appels API au Niveau du Composant (État Local)

Lorsqu'une donnée n'a besoin d'être affichée que dans un seul composant (et non partagée globalement), la requête réseau est déclenchée lors du montage du composant à l'aide du Hook `useEffect`.

2.1 Les Trois États de l'Asynchronisme

Toute requête réseau implique un délai. Une interface utilisateur robuste doit refléter ce délai pour ne pas laisser l'utilisateur dans l'incertitude. Il est impératif de gérer trois états distincts :

1. **La donnée (`data`) :** Initialisée à `null` ou un tableau vide `[]`.
2. **L'état de chargement (`isLoading`) :** Initialisé à `true` ou `false` selon si la requête démarre immédiatement au montage.
3. **L'erreur (`error`) :** Initialisée à `null`, elle stocke le message d'erreur en cas d'échec (panne réseau, erreur 404/500).

2.2 Implémentation Complète avec `useEffect`

JavaScript

```
import { useState, useEffect } from 'react';

export default function StationList() {
  const [stations, setStations] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    // Déclaration de la fonction asynchrone à l'intérieur de l'effet
    async function loadData() {
      setIsLoading(true);
      setError(null);

      try {
        const response = await fetch('/api/stations');
        if (!response.ok) throw new Error('Échec de la connexion au serveur.');

        const data = await response.json();
        setStations(data);
      } catch (err) {
        setError(err.message);
      } finally {
        // Le bloc finally s'exécute toujours, succès ou échec
        setIsLoading(false);
      }
    }
```

```
    loadData();
  }, []); // Tableau vide : exécution unique au montage

  // Rendu conditionnel selon l'état de la requête
  if (isLoading) return <div className="spinner">Chargement des stations...</div>;
  if (error) return <div className="error-banner">Erreur : {error}</div>;

  return (
    <ul>
      {stations.map(station => (
        <li key={station.id}>{station.name}</li>
      ))}
    </ul>
  );
}
```

3. Appels API et État Global : Redux Thunk

Lorsque les données de l'API doivent être partagées entre plusieurs vues (par exemple, la liste des stations dans le tableau de bord et dans l'onglet des alertes), la requête réseau ne doit plus être gérée dans un composant local, mais directement dans **Redux**.

Cependant, les réducteurs (Reducers) Redux doivent rester des fonctions **pures et synchrones**. Pour y intégrer de l'asynchrone, Redux Toolkit fournit un outil natif : `createAsyncThunk`.

3.1 Création d'une Action Asynchrone (Thunk)

`createAsyncThunk` génère une action spéciale qui gère automatiquement les différents cycles de vie de la promesse (en attente, réussie, rejetée).

JavaScript
```
// src/store/stationsSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

// 1. Création de l'action asynchrone
export const fetchStations = createAsyncThunk(
  'stations/fetchStations', // Préfixe de l'action
  async (_, thunkAPI) => {
    const response = await fetch('/api/stations');
    if (!response.ok) throw new Error('Serveur injoignable');
    return await response.json(); // Devient action.payload en cas de succès
```

```
  }
);
```

3.2 Gestion des États dans le Slice (`extraReducers`)

Dans votre `createSlice`, vous devez réagir aux trois étapes de cette requête (pending, fulfilled, rejected) en utilisant la propriété `extraReducers`.

JavaScript

```
const stationsSlice = createSlice({
  name: 'stations',
  initialState: {
    list: [],
    status: 'idle', // 'idle' | 'loading' | 'succeeded' | 'failed'
    error: null
  },
  reducers: {
    // Reducers synchrones classiques ici...
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchStations.pending, (state) => {
        state.status = 'loading';
        state.error = null;
      })
      .addCase(fetchStations.fulfilled, (state, action) => {
        state.status = 'succeeded';
        // Ajout des données reçues depuis l'API dans le store
        state.list = action.payload;
      })
      .addCase(fetchStations.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.error.message;
      });
  }
});

export default stationsSlice.reducer;
```

3.3 Déclenchement (Dispatch) depuis le Composant

Le composant n'a plus qu'à déclencher (dispatch) le Thunk au montage. L'état global se met à jour, et tous les composants abonnés via `useSelector` se synchronisent automatiquement.

JavaScript
```
import { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchStations } from '../store/stationsSlice';

export default function App() {
  const dispatch = useDispatch();
  const { status, error } = useSelector(state => state.stations);

  useEffect(() => {
    if (status === 'idle') {
      dispatch(fetchStations());
    }
  }, [status, dispatch]);

  if (status === 'loading') return <p>Initialisation du système...</p>;
  if (status === 'failed') return <p>Panne critique : {error}</p>;

  return <Dashboard />;
}
```

4. Bonnes Pratiques et Annulation (AbortController)

Un problème classique en React survient lorsque l'utilisateur quitte une page (le composant se démonte) alors qu'une requête HTTP est toujours en cours. Lorsque la requête se termine, elle tente de mettre à jour l'état d'un composant qui n'existe plus, créant une fuite de mémoire.

Pour éviter cela, il faut utiliser l'API native `AbortController` du navigateur pour annuler la requête lors du nettoyage de l'effet.

JavaScript
```
useEffect(() => {
  const controller = new AbortController();
  const signal = controller.signal;

  async function fetchData() {
    try {
      // On passe le signal à fetch
```

```
      const response = await fetch('/api/data', { signal });
      const data = await response.json();
      setData(data);
    } catch (err) {
      // Ignorer l'erreur si elle est due à notre annulation
      if (err.name !== 'AbortError') {
        setError(err.message);
      }
    }
  }

  fetchData();

  // Fonction de nettoyage exécutée au démontage
  return () => {
    controller.abort(); // Annule la requête en cours
  };
}, []);
```

5. Synthèse

1. **Cycle de la donnée réseau :** Toujours prévoir et typer trois états distincts : chargement en cours, succès avec données, ou échec avec erreur.
2. **Local vs Global :** Utiliser `useEffect` + `useState` pour des requêtes isolées. Utiliser Redux (`createAsyncThunk`) pour des données vitales partagées entre plusieurs vues.
3. **Nettoyage :** L'API `AbortController` permet de détruire proprement les requêtes orphelines lorsque la navigation de l'utilisateur change de contexte.